# Technical Document

**Service booking systems: Keyloop and its dealerships, and the design of the Unified Service Scheduler**

| | |
|---|---|
| Version | 2.2 (English edition) |
| Date | 25 October 2026 |
| Based on | The shared Vietnamese working document "Khảo sát hệ thống đặt lịch bảo dưỡng – Keyloop & đại lý" (revision 14) |
| Changes from the Vietnamese edition | Translated into English, and updated to the delivered build: the backend now runs on Scala 3 + Play 3, so the sections on the first code version, concurrency and the SOLID review describe the Scala code. 2.1 adds the features built afterwards (holds, workshop controller, parts, opening hours, package catalogue, tracing), scaling out (5.4) and delivery (4.5). 2.2 adds the second readability and SOLID review (6.4) and brings the diagrams up to date with those features |

## 1. Summary

Two UK dealerships that use Keyloop (Wilsons Epsom and Busseys) both take service bookings through the same multi-tenant SaaS system at `<dealership>.servicestorefront.co.uk`, most likely Keyloop Service Booking. Their home websites were built by two different agencies, so the similar look of the booking pages is an industry convention rather than shared code.

Keyloop's public API schedules work by **service advisor**. Scenario A asks for something deeper: check that a **service bay** and a **technician with the right skill** are both free. That is where this solution goes further than the existing product.

The delivered solution is a working system: a Scala 3 / Play 3 REST API on PostgreSQL, where three database constraints make double booking impossible, and a React web app for seven roles. It is covered by backend unit, property and integration tests, Selenium end-to-end tests in Chrome and a 1000-request load test.

## 2. Agreed decisions

Scenario A is built with the backend in Scala + Play 3 + PostgreSQL. The scheduling core stays in one module so that double booking is prevented inside one transaction.

| Topic | Decision | Reason | Outcome in the build |
|---|---|---|---|
| Scenario | Scenario A, Unified Service Scheduler | Real experience of booking motorbike maintenance in Vietnam | Built |
| Fully implemented layer | Backend; client mocked with the OpenAPI spec and cURL | The backend is the strongest skill | Built. A working React app was added on top, so the whole journey can be demonstrated |

| Topic | Decision | Reason | Outcome in the build |
|---|---|---|---|
| Language | Scala (Java as fallback) | Few candidates use it, strong type safety; Keyloop does not mandate one stack | Scala 3.3.6 LTS (also builds and passes on 3.9.0 and 3.10.0) |
| Framework | Play Framework 3 (on Pekko) | Already familiar; maintained (3.0.11 released May 2026) | Play 3.0.12 with compile-time dependency injection |
| Database | PostgreSQL + Flyway for migrations | Needs exclusion constraints and strong transactions | PostgreSQL 17, Flyway 11 |
| Double-booking protection | `tstzrange` exclusion constraints + `btree_gist` on bay and on technician; test with 50 parallel requests, only the free capacity succeeds | Enforced in the database, correct even with concurrent requests | Three constraints: bay, technician **and vehicle**. Proved by the 50-request test and the 1000-request load test |
| Database access | Slick or Doobie (open) | Not yet chosen | **Anorm** with HikariCP: plain SQL with typed parameters, because the constraints and range queries are PostgreSQL-specific anyway |
| API contract | Tapir on Play, generating OpenAPI and Swagger UI (open) | Meets the API-contract requirement without hand-writing | **Hand-written OpenAPI 3.1** (`conf/openapi.yaml`) served with Swagger UI at `/docs`; a test fails if a route is missing from the contract |
| Architecture | Hexagonal per module (light "Hive"): Scheduling, Catalog, Customer & Vehicle | Boundaries follow data-consistency boundaries; the compiler checks wrong imports | Seven modules (scheduling, consultation, vehicles, catalog, identity, admin, system) in one sbt project; `ArchitectureSpec` enforces the dependency rule |
| Scheduling boundary | Appointment, ServiceBay, Technician, Skill in one module | The no-overlap invariant needs one transaction; splitting would need a saga | As decided |
| Multi-dealership | Every row carries `dealershipId` | Matches the multi-tenant model of servicestorefront | As decided; tests check that staff of one dealership cannot see or change another |
| Extra features | Alternative slots when full; idempotency key for `POST /appointments`; optional HOLD → CONFIRM | Product thinking; ready for AI agents to call | All three built: alternatives, idempotency key, and HOLD → CONFIRM with a 10-minute expiry |
| Chatbot / LLM | Not built; recorded as future work, linked to Keyloop's AIME | Outside the brief, dilutes the core | As decided |
| Batch | No batch framework; expired HOLDs filtered in the query; Pekko Scheduler for any small job | The brief has no real batch processing | Expired holds are ignored in availability queries and released by a small sweeper on a virtual thread (`HoldSweeper`, every 30 s, `FOR UPDATE SKIP LOCKED` so several instances never fight) |

| Topic | Decision | Reason | Outcome in the build |
|---|---|---|---|
| Testing | ScalaTest + ScalaCheck (property-based, no-overlap invariant) + Testcontainers | Verifies AI-written code better than a few unit tests | ScalaTest + ScalaCheck; **embedded PostgreSQL (Zonky)** instead of Testcontainers, so tests need no Docker but still run on the real engine |
| Observability | Structured logs with correlation ID; OpenTelemetry; metric `booking_conflicts_total` | Actually implemented, not just listed | JSON logs with request ID, Prometheus metrics including `booking_conflicts_total`, audit trail, and OpenTelemetry tracing (request, booking attempt and SQL spans, exported to Jaeger over OTLP) |
| Sample data | Seed makes and models from NHTSA vPIC; optional VIN check with timeout and fallback | Free API, no key; booking must not depend hard on an outside service | Own Vietnamese seed data; VIN format checked locally. vPIC not used |
| Working with AI | Write the spec and invariants first; log in `docs/ai-log.md` every AI mistake and its fix; small commits | AI Engineering & Verification carries the most weight | 12 entries in `docs/ai-log.md` |

# 3. Product scope

The Priority column was a proposal to meet the Tuesday 27 October deadline. Everything stays in the design documents; the column only says whether it has code.

## 3.1 User roles

| Role | What they do | Source | Priority | Built |
|---|---|---|---|---|
| System manager | Runs the whole platform, creates and manages dealerships | Original | Future work | **Yes** (onboard and monitor dealerships) |
| Admin (dealership) | Configures bays, technicians, skills, service packages and opening hours of one dealership | Original | Submission | Yes (bays, technicians, staff, opening hours; the package catalogue is managed by the system manager) |
| Sales | Advises on customisation, sells parts and extra services | Original | Future work | Covered by the service advisor role |
| Technician | Sees assigned jobs, updates job status | Original | If time allows | **Yes** |
| Customer (added) | Registers vehicles, books, views and cancels, sends advice requests | servicestorefront booking flow | Submission | Yes |
| Service advisor (added) | Receives customers, advises before booking, confirms bookings | Keyloop API is built around advisors | Submission | Yes |

| Role | What they do | Source | Priority | Built |
|---|---|---|---|---|
| Workshop controller (added) | Sees workshop load, moves a job to another bay or technician | Tribe AI case study: workshop administrators schedule technicians | Future work | **Yes** (workload view per bay and technician; move a job, checked by the same constraints) |
| Parts staff (added) | Handles parts orders | Keyloop Parts Catalogue API has a Parts Advisor role | Future work | **Yes** (stock list and parts orders) |

Simplification adopted: Sales is merged into Service advisor, because both advise before booking.

### 3.2 Features

| Feature | Description | Flow | Priority | Built |
|---|---|---|---|---|
| Sign in, sign out, permissions | Each role can only call its own APIs; data separated by `dealershipId` | With JWT, sign-out needs short-lived tokens or a revocation list | Submission | Yes, with opaque bearer tokens stored as SHA-256 hashes in `sessions`, so sign-out really revokes |
| Maintenance packages (main feature) | Customers who know little about vehicles pick a ready-made package with a checklist, duration, required skill and bay type | Vehicle → package → dealership → slot → confirm | Submission | Yes |
| Inspection on request, with advice | The customer names what to check; an advisor assesses first, then the booking is made | Request → advisor sets duration and skill → customer picks a slot | If time allows | **Yes** |
| Customisation with advice (secondary) | Same advice flow as inspection on request | As above, plus a quote | Future work | **Yes** |
| Replacement parts ordering | Order parts, optionally attached to an appointment | A separate domain (stock, price, orders); Keyloop has its own Order Parts API | Future work | **Yes**, as its own `parts` module: REQUESTED → ORDERED → RECEIVED → ISSUED; issuing locks the stock rows and a `CHECK (stock_qty >= 0)` stops two people issuing the last unit |

Inspection on request and customisation share one mechanism, "advice request → appointment". Building it once served both.

## 4. Implementation

### 4.1 History

The first version (24 October) was written in TypeScript (Node + Fastify) because the first build environment could not download Scala or Play libraries (Maven Central was blocked). The schema, the hexagonal structure and the double-booking constraints were designed to be portable. On 25 October, with Maven Central reachable, the backend was ported to **Scala 3 + Play 3** with the same modules,

class names and ports, and every test was ported case by case with the same expected values. The TypeScript backend was then removed.

## 4.2 What is built

- **Roles:** sign-in and sign-out for seven roles (Customer, Service advisor, Technician, Dealership admin, System manager, Workshop controller, Parts staff).
- **Customer:** maintenance packages, inspection on request and customisation through the advice flow, appointments, cancellation, vehicles, add to calendar. The chosen time is **held for 10 minutes** (HELD) while the customer reviews it, with a countdown, then confirmed (CONFIRMED).
- **Staff:** the advisor's workshop-day board, the technician's job list, management of bays, technicians, staff and opening hours, dealership onboarding, the service package catalogue, the workshop controller's workload view and job moves, parts stock and parts orders.
- **Double-booking protection:** three exclusion constraints (bay, technician, vehicle); retry after a lost race; idempotency key; three nearest alternatives when a slot has just been taken.
- **Logging:** JSON logs with a request ID that flows from the browser to the server, Prometheus metrics, an audit trail of appointment status changes, OpenTelemetry traces.
- **Delivery:** Docker images, a scaled-out docker-compose profile, a GitHub Actions pipeline and Azure Container Apps (see 4.5).
- **Not built:** rescheduling (today: cancel and rebook), SMS / Zalo notifications, breaks and holidays in shifts, multi-technician jobs.
- **AI log:** `docs/ai-log.md` records 12 real defects caught in AI-written code, for the AI Collaboration Narrative.

## 4.3 Stack

| Layer | Technology |
|---|---|
| Language and runtime | Scala 3.3.6 (LTS), JDK 21 (virtual threads) |
| Web framework | Play Framework 3.0.12 on Pekko HTTP, compile-time DI (`AppLoader`) |
| Database | PostgreSQL 17, Flyway 11.20 migrations, Anorm 2.7, HikariCP 6.3 |
| Embedded database | Zonky embedded-postgres (PostgreSQL 17.6 binaries) for `sbt run` and tests |
| JSON | play-json, Jackson pinned to 2.19.1 |
| Logging and metrics | Logback with logstash-logback-encoder, Prometheus Java client 1.10 |
| Tracing | OpenTelemetry SDK with the OTLP exporter (Jaeger in docker-compose) |
| API documentation | OpenAPI 3.1, Swagger UI 5.17 |
| Tests | ScalaTest, ScalaCheck; Selenium WebDriver for end-to-end tests (`e2e/`) |
| Frontend | React 19, React Router 7, TanStack Query 5, Vite 8, TypeScript 6, Luxon |

## 4.4 How a booking stays correct

1. `GET /api/availability` computes, for each start time on the dealership's grid, how many (bay, technician) pairs are free for the whole job: a bay of the required type, and a technician with the required skill who is on shift. This is pure code in `domain/Availability.scala`.
2. `POST /api/appointments/holds` (the web app) or `POST /api/appointments` checks again inside a transaction, releases holds that ran out in its way, picks a pair and inserts (HELD for 10 minutes, or CONFIRMED). `POST /api/appointments/:id/confirm` turns a running hold into a booking. **Best fit:**

the technician with the fewest skills is chosen first, so specialists stay free for work only they can do; ties go to whoever has the least booked that day.
3. PostgreSQL is the final judge. Three `EXCLUDE USING gist` constraints on `tstzrange` periods make overlapping bookings of the same bay, technician or vehicle impossible. Ranges are half-open, so 09:00–10:00 and 10:00–11:00 do not clash.
4. If a concurrent request won the same pair (SQLSTATE `23P01`), the service retries with fresh data, up to five times. If nothing is left, it answers `409 SLOT_UNAVAILABLE` with the nearest alternatives.
5. An `Idempotency-Key` header makes retries and double taps safe: the same key returns the original appointment (`200`, `Idempotent-Replayed: true`). An advisory lock per customer and key queues duplicate requests instead of letting them race.

The data model is drawn in `diagrams/erd.png`: business tables in green, authentication and authorisation tables in red, framework tables in orange.

### 4.5 Delivery: containers, CI/CD and Azure

| Piece | File | What it does |
|---|---|---|
| API image | `backend/Dockerfile` | sbt builds and stages the Play app; the runtime image is a Java 21 JRE running as a non-root user, with a health check on `/health` |
| Web image | `frontend/Dockerfile`, `frontend/nginx.conf.template` | Vite build served by nginx, which also forwards `/api`, `/docs` and `/health` to the API, so the browser sees one origin and needs no CORS |
| Scaled-out local run | `docker-compose.yml` (profile `full`) | 3 API instances, nginx, one PostgreSQL, Jaeger and Prometheus (which finds every API instance by DNS) |
| End-to-end tests | `e2e/` | Selenium (Scala, ScalaTest) drives Chrome through the web app: one booking across five roles (customer, advisor, workshop controller, technician, parts staff), sign-in rules, and the back office (admin, system manager). 26 tests, run in CI against the built images |
| Pipeline | `.github/workflows/ci-cd.yml` | Every pull request: backend tests, web lint and build, both images built, end-to-end tests against them. Push to `master`: images pushed to Azure Container Registry, rolled out to Azure Container Apps, then the live site is smoke-tested |
| Azure set-up | `deploy/azure/setup.sh` | One script for Cloud Shell: registry, PostgreSQL Flexible Server, the Container Apps environment and both apps, and a managed identity that GitHub signs in as with OpenID Connect (no stored password) |

On Azure the API is internal only and scales from 1 to 5 replicas on concurrent HTTP requests; the web app is the only public endpoint (HTTPS). The region is Japan East, one of the regions an Azure for Students subscription allows.

## 5. Concurrency and load

The system answers all 1000 requests fired at the same instant, with no errors and no double booking. It uses **Java 21 virtual threads**.

## 5.1 Concurrency model

- **Non-blocking HTTP, blocking code on virtual threads:** Play's HTTP threads never block. Each use case runs on its own virtual thread (`Blocking` in `http/Http.scala`). JDBC calls and password hashing block that virtual thread, which costs a few hundred bytes, not an operating system thread, so 1000 waiting requests cost memory, not 1000 threads.
- **Real parallelism is in the database:** each API instance has a pool of 20 PostgreSQL connections (HikariCP). Nested calls on one thread reuse the same connection, so a burst cannot starve the pool.
- **Correctness does not depend on threads:** the no-double-booking rule is enforced by three PostgreSQL `EXCLUDE` constraints, so it holds for any number of instances.
- **Scaling out:** run more API instances behind a load balancer. The API keeps no state in memory; sessions are in the database.

## 5.2 Load test results

`sbt loadTest` (or `npm run test:load`) starts the API on a real socket and fires 1000 requests at once over HTTP, three times. Measured on 25 October 2026 on a 2-vCPU machine, with the API, PostgreSQL and the load generator sharing the two cores.

| Burst of 1000 requests | Result | Wall time | p50 / p99 |
|---|---|---|---|
| Same slot, 4 free bay + technician pairs | exactly 4 × 201, 996 × 409 with alternatives, 0 errors | 5.5 s | 4.4 s / 5.3 s |
| Spread over 54 slots, 20 pairs | 1000 × 201, 0 overlaps in the database | 3.2 s | 2.2 s / 3.0 s |
| Availability reads | 1000 × 200 | 2.1 s | 1.6 s / 2.0 s |

Latency here is queueing on two shared cores, not a limit of the design.

## 5.3 Problems found by load testing (fixed)

| Problem | Cause | Fix | Before → after |
|---|---|---|---|
| Pressing Book repeatedly with one Idempotency-Key: in about 1 run in 4, some requests got 500 | PostgreSQL broke a deadlock (`40P01`) between the unique-key check and the exclusion constraint | Advisory lock per customer + key so the requests queue; deadlocks are still retried as a safety net; regression test runs 10 rounds and fails without the fix | test: 14 s → 0.4 s |
| 1000 availability reads took 43 s (first version) | About 40% of CPU was time-zone arithmetic per technician per slot | Compute each slot's time window once, use a fixed offset on days without a clock change, compute booked minutes outside the sort | 43 s → 4.9 s |
| Scala port: requests hung for 30 s and failed under a burst | Connection-pool starvation: a use case held a connection while a nested port call waited for a second one; all 20 were taken | A thread-local session in `PgDb`: nested `run` / `tx` calls reuse the current connection or join the open transaction | 1000 requests answered |

| Problem | Cause | Fix | Before → after |
|---|---|---|---|
| Scala port: the first burst failed with "received no bytes" | Pekko HTTP's accept backlog (default 100) overflowed when 1000 clients connected at once | `pekko.http.server.backlog = 1024` | 0 dropped connections |

For the interview: "Play keeps the HTTP threads non-blocking and runs each use case on a Java 21 virtual thread, so blocking JDBC code stays simple and 1000 waiting requests cost memory, not threads. Correctness under contention lives in database constraints, and a 1000-request load test proves it." Details: the Load test section of the README and entries 6, 7, 11 and 12 in `docs/ai-log.md`.

## 5.4 Scaling out: several instances, one set of rules

Scala's name comes from "scalable"; here scaling means adding instances, not changing code. Nothing that decides correctness lives in an instance's memory:

| Concern | Where it lives | Why more instances are safe |
|---|---|---|
| No double booking | PostgreSQL `EXCLUDE` constraints on bay, technician and vehicle | The database refuses the second booking whichever instance sent it |
| Sign-in | `sessions` table (hashed tokens) | Any instance can check any token; no sticky sessions |
| Idempotency key | Advisory lock per customer + key, unique index | Two instances replaying one key queue on the same lock |
| Expired holds | `HoldSweeper` with `FOR UPDATE SKIP LOCKED` on every instance | Each expired hold is released once; instances skip rows another one is handling |
| Demo data | Advisory lock around the seed | Instances starting together seed once |
| Parts stock | Row locks in id order + `CHECK (stock_qty >= 0)` | The last unit cannot be issued twice |
| Tracing | W3C `traceparent` continued by each instance | One trace follows a request across nginx and any replica |

Every response carries `X-Served-By` (the replica name) and `X-Trace-Id`, so spreading is visible from outside. `scripts/load-test.mjs` checks it against a running deployment. Measured on 25 October 2026 with `docker compose --profile full` (3 API instances, one PostgreSQL, all on one 2-vCPU machine):

| Test | Result |
|---|---|
| 1000 availability reads, 50 at a time, through nginx | 1000 × 200; 332 / 335 / 333 answered by the three instances |
| 15 vehicles book the same 08:00 slot at once (3 free bay + technician pairs) | exactly 3 × 201 and 12 × 409; the requests were spread 5 / 5 / 5 over the instances, so the race was decided by PostgreSQL, not by a lock in one process |

Jaeger showed each booking as one trace: the HTTP span, the `booking.attempt` span and its SQL spans; Prometheus scraped all three instances.

# 6. SOLID review (25 October 2026)

The code meets SOLID after one refactoring pass: the review found eight violations and fixed all eight. The API, the tests and the 1000-request load test gave the same results as before. The dependency rule is checked by an automated test, not only by review. The review was done on the first version; the Scala backend was written with the same structure, so every fix holds there.

| # | Principle | Before | After (Scala names) |
|---|---|---|---|
| 1 | SRP | `BookingService` (403 lines) did availability, booking, read views and status changes | Split into `AvailabilityService`, `BookingService`, `AppointmentService`; shared rules in `WorkshopContext` |
| 2 | SRP | Admin, vehicles and system routes ran SQL inside the handler | Each module has a controller (HTTP), `application/` (use cases), `adapters/Pg*.scala` (SQL), `Ports.scala` (traits) |
| 3 | DIP | `BookingService` created a concrete catalogue repository | It receives a `CatalogReader` trait; only `AppComponents` picks implementations |
| 4 | DIP | Booking and consultation called a vehicle lookup in the vehicles HTTP adapter | Both depend on the `OwnedVehicles` trait |
| 5 | DIP | `BookingService` read PostgreSQL error codes (`23P01`, `40P01`) | Adapters translate them into `SlotTakenError`, `VehicleBusyError`, `TransientStorageError`; SQL codes exist only in the adapter layer |
| 6 | DIP | The database port exposed driver types | `Db` and `DbSession` are our own traits; the application layer never imports JDBC or Anorm |
| 7 | ISP | Booking depended on the whole `ConsultationService`, the guard on the whole `IdentityService`, every use case on an 11-method store | `ConsultationBookingPort` (3 methods), `Authenticator` (1 method), the store split into `ResourceCalendar`, `AppointmentReader`, `AppointmentWriter` |
| 8 | OCP | Status-transition errors mapped by a chain of `if`s | A `TransitionErrors` table: a new rule is one new entry |

## 6.1 Evidence

- **LSP:** `BookingService` runs on in-memory fakes (`BookingServiceFakesSpec`, no database) and on PostgreSQL (integration tests) with the same expectations.
- **DIP is enforced:** `ArchitectureSpec` fails if a domain, application or port file imports an adapter, the HTTP layer, `play.api.mvc`, Anorm, JDBC, the driver or the SQL helpers. A deliberate violation was tried to confirm the test catches it.
- **Checked:** `-Wunused:all -Werror` compiles clean; 95 tests passed at the time of that review; the 1000-request load test passes; the packaged server boots and the web app runs every page in a headless browser with no errors.

## 6.2 Bugs found during the review (fixed)

| Bug | Fix |
|---|---|
| The slot grid ignored `maxDaysAhead`: a far date looked free, and a suggested alternative could be outside what the dealership accepts, so booking it failed with `TOO_FAR_AHEAD` | One shared `lastBookableDay` rule; a property test checks that every slot shown as free can be booked (it fails without the fix) |

| Bug | Fix |
|---|---|
| A job the dealership cannot do answered `409` with an empty list | Now `422 NOT_OFFERED_HERE`, documented in the OpenAPI contract |
| A deadlock outside the booking flow would have answered `500` | Now `503 TRY_AGAIN` with `Retry-After` |

### 6.3 Kept on purpose

Metrics and the logger are shared across the system; request shapes (play-json `Reads`) sit next to the use cases; ports take a `DbSession` so one use case can write several tables in one transaction; password hashing is a plain object; seed, migrations and `/health` run SQL directly because they are infrastructure tools. Details in `docs/solid-review.md`.

### 6.4 Second review (26 October 2026)

The features added after the first review (holds, workshop controller, parts, opening hours, package catalogue, tracing) were reviewed for readability and SOLID. The structure held: every new feature follows the module layout (controller → service → port → `Pg*` adapter), the new rules are tables (`Lifecycle.Transitions`, `PartOrderLifecycle`), database errors are still translated in adapters (`OutOfStockError` joins `SlotTakenError` and `VehicleBusyError`), and `ArchitectureSpec` still passes. No violation of the kind fixed in 6 came back.

Smaller findings, none of which changes behaviour:

| Finding | Principle | Where |
|---|---|---|
| `BookingService.book` is about 110 lines with two nested closures | SRP, readability | `BookingService.scala` |
| "The caller's dealership or 403" is written five times; releasing expired holds three times (`DispatchService` does not count the `holds{expired}` metric) | DRY | scheduling, admin, parts services |
| `WorkshopAdminService` and `TenantService` take the concrete `IdentityService` instead of a narrow trait | DIP, ISP | admin, system |
| `consultation` and `scheduling` import each other (`BookableConsultation` lives in consultation, the port in scheduling) | Module boundaries | both modules |
| `BookPage.tsx` is 594 lines with 13 pieces of state | SRP (frontend) | `frontend/src/pages/customer/BookPage.tsx` |

The full list (13 findings with suggested fixes) is in `docs/code-review-2026-10-26.md`. The same review found the diagrams and the SRS still described the first release; they were redrawn and updated (HELD and EXPIRED in the state diagram, use cases UC12, UC34, UC35, UC42, UC53, UC62, UC70, UC71, the parts tables in the ERD and domain model, the new services in the class and hexagonal diagrams).

## 7. Websites surveyed

Both dealerships point their "Book Service" button at the same platform, with a different subdomain each.

| Site | Role | Technology observed | Note |
|---|---|---|---|
| wilsons.co.uk | Wilsons Epsom (Surrey) home site | NetDirector, footer links to keyloop.com; images from images.netdirector.co.uk; static files on AWS CloudFront | "Service & MOT" leads to the booking site |
| wilsons.servicestorefront.co.uk | Wilsons booking | Could not be opened with the HTML reading tool | According to Keyloop, Wilsons takes about 500 bookings a month through Service Booking |
| busseys.co.uk | Busseys (Norfolk) home site | WordPress, built by the agency Netmatters | Not a Keyloop platform |
| busseys.servicestorefront.co.uk | Busseys booking | Could not be opened with the HTML reading tool | Busseys is a Service Hub pilot site |

The `<dealership>.servicestorefront.co.uk` pattern shows one system shared by many dealerships, one subdomain each. It is not 100% confirmed to be a Keyloop product, because the pages could not be opened from the research environment.

## 8. Keyloop's Service Appointment Booking API

The documentation is public on Keyloop Developer, but calling the API requires joining the Partner Programme and paying per call. It runs on the Autoline, Aswin, Automaster, Autoline Drive and WinDrakkar DMSs.

| Method | Endpoint | Purpose |
|---|---|---|
| GET | `/appointments/service-categories` | Service categories |
| GET | `/appointments/service-products` | Specific services |
| GET | `/appointments/options` | Booking options (for example a courtesy car) |
| GET | `/advisors`, `/advisors/{advisorId}` | Service advisors |
| GET | `/advisors/availability` | Advisors' free time |
| GET | `/advisor-groups` | Advisor groups |
| POST | `/appointments/availability` | Find free slots |
| POST | `/appointments/options/availability` | Free slots for an option |
| POST | `/appointments/service-products/availability` | Free slots for a service |
| POST | `/appointments` | Create an appointment |
| DELETE | `/appointments/{appointmentId}` | Cancel an appointment |

Notable rules:

- Free slots are checked in real time against the workshop diary in the DMS.
- An appointment can only be cancelled while the dealership has not started work on the vehicle.
- No endpoint deals with bays or technician skills.

# 9. Related products

| Product | Owner | What it does | Relevance to Scenario A |
|---|---|---|---|
| Service Hub | Keyloop | Cloud after-sales platform: online booking, check-in, booking management, daily dispatch, workshop management | The closest real product to Scenario A, in the Ownership domain |
| Tribe AI scheduling engine | Keyloop + Tribe AI | Builds a near-optimal technician plan every morning by job type, skill and bay | Shows Keyloop treats bay and technician scheduling as a real problem |
| AIME | Motortech.ai, acquired by Keyloop | AI agent that talks to customers and books appointments | Mainly sales; not in scope |
| Autoflows | Third party | AI voice agent takes slots from Keyloop and writes confirmed bookings back | An example of an AI agent calling a booking API |
| TruVideo in Service Hub | Keyloop + TruVideo | Vehicle inspection videos attached to the service record | Out of scope |
| UVeye | UVeye | Automatic vehicle scan in the service lane | Out of scope; context for the video |

# 10. Improvement opportunities for Scenario A

These points come from public documentation and were not verified on the live interface.

| Observed today | Improvement proposed | Built |
|---|---|---|
| Free slots are computed per advisor | Check at the same time that a bay and a technician with the right skill are free for the whole job | Yes |
| Double-booking protection depends on the DMS | Enforce it in the database with PostgreSQL exclusion constraints, proved by a concurrency test | Yes |
| When a slot is full, the customer searches for another time alone | Return the few nearest free slots when the requested time is taken | Yes |
| No visible hold while the customer fills in the form | A temporary hold with an expiry (HOLD → CONFIRM) | Yes (10 minutes, countdown in the booking page) |
| No visible protection against pressing Book twice | Idempotency key for `POST /appointments`, useful also for AI agents such as AIME or Autoflows | Yes |
| Cancellation only before work starts | Keep the rule and model the appointment states explicitly | Yes (state machine in `domain/Lifecycle.scala`) |
| Multi-tenant through subdomains | Tie every row to `dealershipId` from the start | Yes |

## 11. Open checks to do by hand

- [ ] Open both servicestorefront sites, walk the booking flow up to the confirmation step, and take a screenshot of each step.
- [ ] Open F12 → Network to see which domain the booking API calls go to.
- [ ] Look for a "Powered by" line or the Keyloop name on the booking pages.
- [ ] Watch the Service Hub webinar on keyloop.com to see the workshop-side interface.

## 12. Sources


- Wilsons Epsom
- Busseys
- Keyloop – Wilsons Epsom testimonial
- Keyloop – Service Hub launch
- Keyloop Developer – Products
- Keyloop Developer – Service Appointment Booking
- Keyloop – Product-specific terms (API plans)
- Tribe AI – Keyloop case study
- IT Brief UK – Keyloop buys Motortech.ai
- Autoflows – Keyloop integration